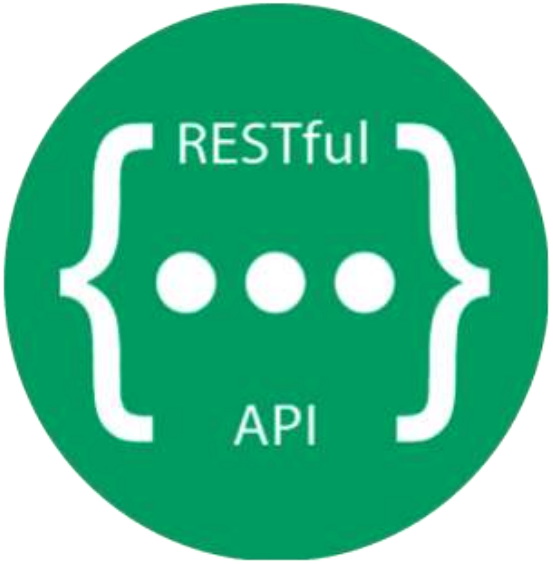

Create

Read

Update

Delete

.NET Core Web API interview questions and answers

1. What is .NET Core?

Answer:
.NET Core (now part of **.NET**) is a **cross-platform, open-source framework** developed by Microsoft for building modern applications like web apps, APIs, microservices, and cloud apps.

2. What is ASP.NET Core Web API?

Answer:
It is a framework for building **RESTful services (APIs)** using HTTP methods like GET, POST, PUT, DELETE. It allows communication between client and server.

3. What is REST?

Answer:

REST (Representational State Transfer) is an **architectural style** that uses HTTP protocols to perform CRUD operations.

4. What are HTTP Methods?

Answer:

GET – Retrieve data

POST – Create data

PUT – Update data

DELETE – Remove data

5. What is a Controller?

Answer:

A Controller handles incoming HTTP requests and returns responses.

Example:

```
[ApiController]
[Route("api/[controller]")]
public class EmployeeController : ControllerBase
{
[HttpGet]
public IActionResult Get()
{
return Ok("Hello World");
}
}
```

6. What is Dependency Injection (DI)?

Answer:

DI is a design pattern used to **inject dependencies into classes**, improving testability and loose coupling.

Example:

```
public class EmployeeService
{
public string GetName() => "John";
}
```

Register in Program.cs:

```
builder.Services.AddScoped<EmployeeService>();
```

6. What is IActionResult?

Answer:

IActionResult is a return type that allows you to return different HTTP responses like:

Ok()

NotFound()

BadRequest()

8. What is Middleware?

Answer:

Middleware are components that handle requests and responses in the pipeline.

Example:

app.UseAuthorization();

9. What is appsettings.json?

Answer:

It is a configuration file used to store settings like:

Connection strings

API keys

Logging config

10. What is Routing?

Answer:

Routing maps incoming HTTP requests to controllers/actions.

Example:

```
[Route("api/[controller]")]
```

11. What is Model Binding?

Answer:

It automatically maps HTTP request data to action method parameters.

Example:

```
public IActionResult Create(Employee emp)
```

12. What is JSON?

Answer:

JSON (JavaScript Object Notation) is a lightweight format used for data exchange between client and server.

13. What is Swagger?

Answer:

Swagger is used for **API documentation and testing UI**.

14. What is Entity Framework Core?

Answer:

Entity Framework Core is an ORM (Object Relational Mapper) that helps interact with the database using C# instead of SQL.

15. What is DbContext?

Answer:

It represents a session with the database and allows querying and saving data.

16. What is CORS?

Answer:

CORS (Cross-Origin Resource Sharing) allows APIs to be accessed from different domains.

17. What is Status Code?

Answer:

200 → OK

201 → Created

400 → Bad Request

404 → Not Found

500 → Server Error

18. What is Program.cs in .NET Core?

Answer:

It is the entry point where application services and middleware are configured.

19. Difference between .NET Core and .NET Framework?

Answer:

Feature	.NET Core	.NET Framework
Platform	Cross-platform	Windows only
Performance	High	Moderate
Open Source	Yes	No

20. What is API Versioning?

Answer:

It helps maintain multiple versions of APIs (v1, v2) without breaking existing clients.

🚀 Expert-Level .NET Core Web API Interview Q&A

1. How does the ASP.NET Core request pipeline work internally?

Answer:

ASP.NET Core uses a **middleware pipeline** where each middleware:

Receives HttpContext

Can process the request

Calls the next middleware or short-circuits

Flow:

Client → Kestrel Server → Middleware → Routing → Endpoint → Response

Key points:

Order of middleware matters

Built using RequestDelegate

Supports async processing

2. What is Kestrel and how does it differ from IIS?

Answer:

Kestrel: Cross-platform, high-performance web server (default in ASP.NET Core)

IIS: Windows-based web server used as reverse proxy

Best practice:

Use Kestrel behind IIS or Nginx in production

3. Explain Dependency Injection lifetimes in detail

Answer:

Lifetime	Description	Use Case
Transient	New instance every time	Lightweight services
Scoped	One per request	DB context
Singleton	One for app lifetime	Caching

4. How do you handle global exception handling?

Answer:

Use middleware or filters.

Example:

```
app.UseExceptionHandler("/error");
```

Or custom middleware:

```
public class ExceptionMiddleware
{
private readonly RequestDelegate _next;
public ExceptionMiddleware(RequestDelegate next)
{
_next = next;
}

public async Task Invoke(HttpContext context)
{
try
{
await _next(context);
}
catch (Exception ex)
{
context.Response.StatusCode = 500;
await context.Response.WriteAsync("Error occurred");
}
}
}
```

5. How do you implement API versioning?

Answer:

Options:

URL versioning → /api/v1/products

Query string → ?version=1

Header versioning

Use package:

Microsoft.AspNetCore.Mvc.Versioning

6. How do you secure APIs?

Answer:

JWT Authentication

OAuth2

Role-based authorization

Policy-based authorization

Example:

```
[Authorize(Roles = "Admin")]
```

7. What is JWT authentication flow?

Answer:

User logs in

Server generates token

Client sends token in header

Server validates token

Header:

Authorization: Bearer <token>

8. How do you improve API performance?

Answer:

Use async/await

Response caching

Compression

Pagination

Avoid blocking calls

Use IQueryable properly

Minimize over-fetching

9. What is the difference between IEnumerable and IQueryable?

Answer:

Feature	IEnumerable	IQueryable
Execution	In-memory	Deferred (DB)
Performance	Slower	Faster for DB queries

10. What is EF Core tracking vs no-tracking?

Answer:

Tracking: EF tracks entity changes (default)

NoTracking: Faster, read-only queries

Example:

```
context.Users.AsNoTracking()
```

11. How do you implement caching?

Answer:

In-memory caching

Distributed caching (Redis)

Example:

IMemoryCache cache;

12. What is rate limiting?

Answer:

Controls number of requests per client to prevent abuse.

Techniques:

IP-based limiting

Token bucket algorithm

13. What are Filters in ASP.NET Core?

Answer:

Types:

Authorization filters

Action filters

Exception filters

Result filters

Used for cross-cutting concerns.

14. What is middleware vs filters?

Answer:

Middleware	Filters
Global pipeline	MVC-specific
Runs before routing	Runs after routing

15. How do you handle large data responses?

Answer:

Pagination

Streaming (IAsyncEnumerable)

Compression

Chunking

16. What is async/await best practice in Web API?

Answer:

Avoid .Result and .Wait()

Use async all the way

Improves scalability (non-blocking threads)

17. What is CQRS pattern?

Answer:

CQRS separates:

Commands (write)

Queries (read)

Benefits:

Scalability

Better performance tuning

18. How do you implement logging?

Answer:

Built-in logging framework from Microsoft:

ILogger<MyClass>

Tools:

Serilog

NLog

19. What is health check in API?

Answer:

Used to monitor API status.

Example:

```
app.MapHealthChecks("/health");
```

20. How do you design a scalable API?

Answer:

Stateless services

Load balancing

Microservices architecture

Caching layer

Database optimization

Use message queues (RabbitMQ, Azure Service Bus)

21. What are common API design best practices?

Answer:

Use nouns in URLs

Proper HTTP verbs

Versioning

Consistent response format

Use status codes properly

22. How do you handle concurrency in EF Core?

Answer:

Use RowVersion / Timestamp

Optimistic concurrency

23. What is HATEOAS?

Answer:

Hypermedia as the Engine of Application State—API responses include links to related resources.

24. How do you deploy .NET Core API?

Answer:

IIS

Docker containers

Cloud (Azure, AWS)

25. Real-world scenario question

Q: Your API is slow in production. What will you check?

Answer:

DB queries (indexes, execution plan)

API response time

Logging

CPU/memory usage

Network latency

Caching opportunities

🔐 JWT Interview Questions & Answers

1. What is JWT?

Answer:
JWT (JSON Web Token) is a **compact, URL-safe token format** used for securely transmitting information between parties as JSON.

It is commonly used for:

Authentication

Authorization

2. Structure of a JWT token?

Answer:

A JWT consists of **3 parts** separated by dots (.):

Header.Payload.Signature

Example:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Parts:

Header – Algorithm & token type

Payload – Claims (user data)

Signature – Ensures integrity

3. What are Claims in JWT?

Answer:

Claims are **key-value pairs** stored in the payload.

Types:

Registered: sub, exp, iss

Public: Custom claims

Private: Application-specific

4. How JWT authentication works?

Answer:

User logs in with credentials

Server validates user

Server generates JWT

Client stores token

Client sends token in request header

Server validates token

Header:

Authorization: Bearer <token>

5. What is the Signature in JWT?

Answer:

Signature ensures the token is **not tampered with**.

Example:

```
HMACSHA256(
base64UrlEncode(header) + "." + base64UrlEncode(payload),
secret
)
```

6. Difference between Authentication and Authorization?

Answer:

Authentication	Authorization
Who you are	What you can access
Login process	Permission check

7. What algorithms are used in JWT?

Answer:

HS256 (HMAC)

RS256 (RSA)

Difference:

HS256 → Symmetric (same key)

RS256 → Asymmetric (public/private key)

8. Where is JWT stored on client side?

Answer:

LocalStorage

SessionStorage

Cookies (HttpOnly recommended)

9. What are advantages of JWT?

Answer:

Stateless authentication

Scalable

Cross-domain support

No server session required

10. What are disadvantages of JWT?

Answer:

Cannot easily revoke tokens

Token size is large

Security risks if stored improperly

11. How do you validate JWT in .NET Core?

Answer:

Using middleware:

```
builder.Services.AddAuthentication("Bearer")
.AddJwtBearer(options =>
{
options.TokenValidationParameters = new
TokenValidationParameters
{
ValidateIssuer = true,
ValidateAudience = true,
ValidateLifetime = true,
ValidateIssuerSigningKey = true
};
});
```

12. What is Token Expiration?

Answer:

JWT has an expiry time using exp claim.

Example:

Short-lived token (15–30 mins)

13. How do you refresh JWT tokens?

Answer:

Use **Refresh Tokens**:

Access token → short life

Refresh token → long life

Generate new access token using refresh token

14. What is Bearer Token?

Answer:

A Bearer token is a token that grants access to resources.

Authorization: Bearer <token>

15. How do you secure JWT?

Answer:

Use HTTPS

Keep secret key safe

Use short expiry

Use refresh tokens

Validate issuer & audience

16. What is role-based authorization in JWT?

Answer:

Roles are stored in claims.

Example:

```
[Authorize(Roles = "Admin")]
```

17. What is policy-based authorization?

Answer:

More flexible than roles.

Example:

```
options.AddPolicy("AdminOnly", policy =>
policy.RequireClaim("role", "admin"));
```

18. What happens if JWT is stolen?

Answer:

Attacker can access API

Mitigation:

- Short expiry
- Refresh token rotation
- IP/device validation

19. JWT vs Session-based authentication?

Answer:

JWT	Session
Stateless	Stateful
Scalable	Less scalable
Stored client-side	Stored server-side

20. Real-world scenario

Q: How do you logout a JWT user?

Answer:

JWT is stateless, so:

Cannot directly invalidate token

Solutions:

- Maintain blacklist
- Use short expiry
- Remove refresh token

🔥 Advanced Discussion Points (Very Important)

✅ Token Revocation Strategies

Blacklisting tokens

Rotating refresh tokens

✅ Microservices Authentication

Use central identity provider

Validate JWT in each service

✅ Best Practice Architecture

Identity Server (Auth server)

API Gateway

Distributed cache